

# CSE2008 – Operating Systems

Dr. Venkata Rami Reddy Ch  
Sr.Assistant Professor  
School of Computer Science & Engineering  
VIT-AP University

- Process and programs, process states, process concept, process scheduling, IPC, multithreading models.

## Program

- It is a set of instructions written in a programming language
- Stored on disk.
- A program is a passive entity.
- It does not perform any action by itself.

## Process

- A process is a program in execution.
- A process is not just the program code but the entire environment necessary for execution.
- A process will need certain resources—such as CPU time, memory, files, and I/O devices to accomplish its task.
- It includes: Program code, Current activity (value of Program Counter, CPU registers), Process state and Memory (stack, heap, data, code)

# Process in Memory

- When a **program becomes a process**, it is loaded into **main memory (RAM)** by the operating system.
- The memory layout of a process is **divided into several segments**, each with a specific purpose.

## Text Section:

- Contains the program's code (instructions).

## Stack:

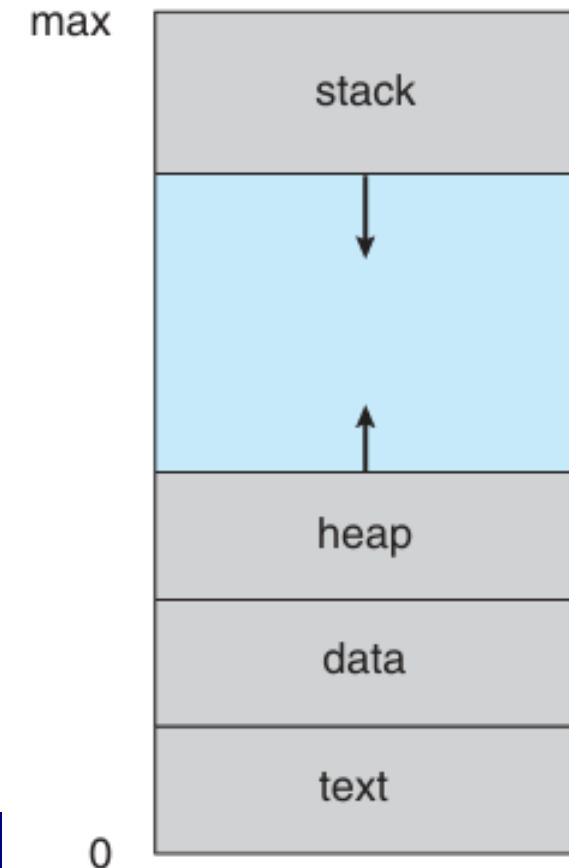
- Used to store **temporary data**:
  - Function parameters
  - Return addresses
  - Local variables

## Data Section:

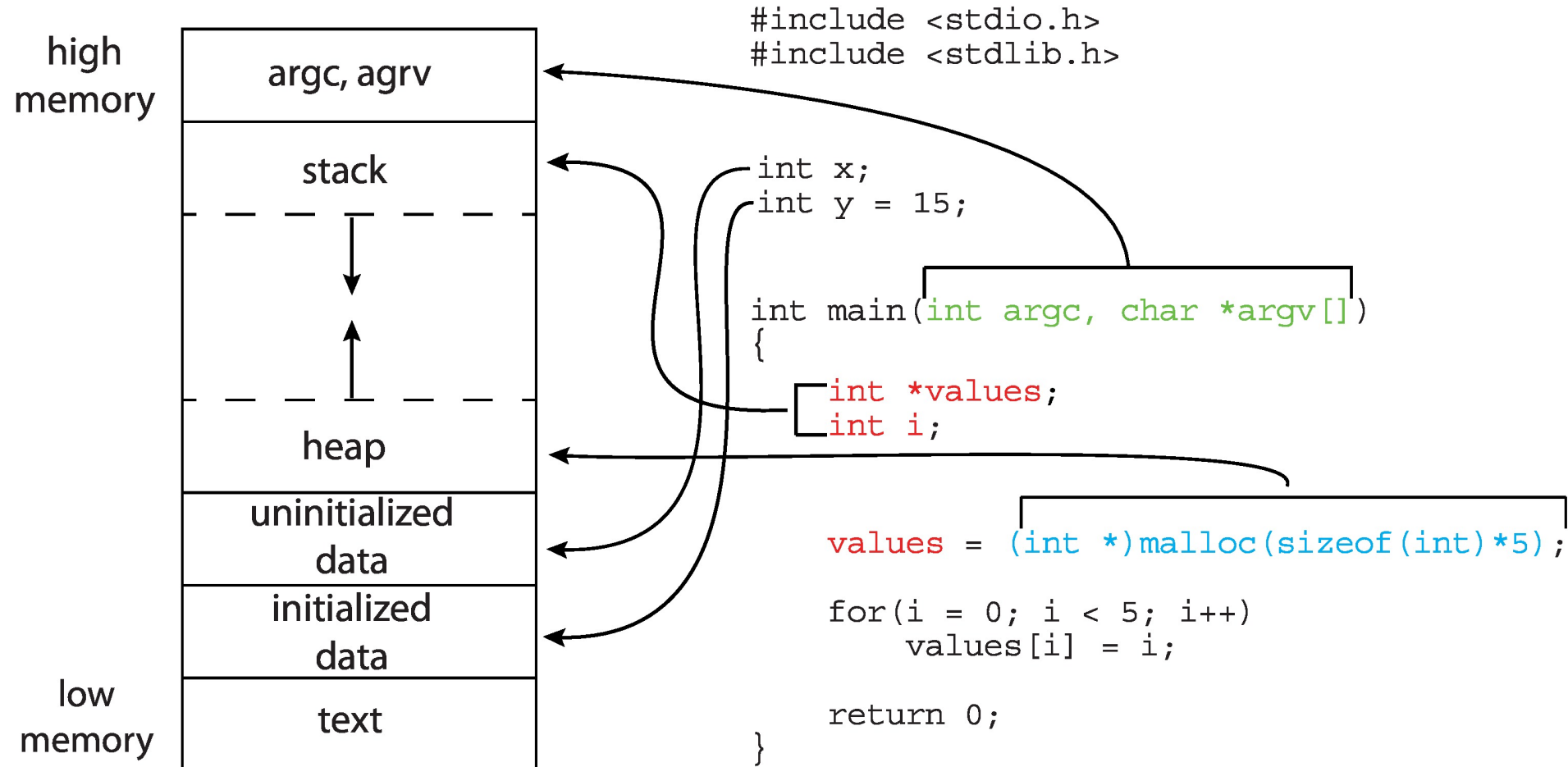
- Stores **global variables**.

## Heap:

- Dynamically allocated memory at **runtime** (e.g., via malloc() in C).



# Memory Layout of a C Program



# Process State

- A **process** moves through various **states** during its lifetime.
- These states help the operating system manage and schedule processes effectively.

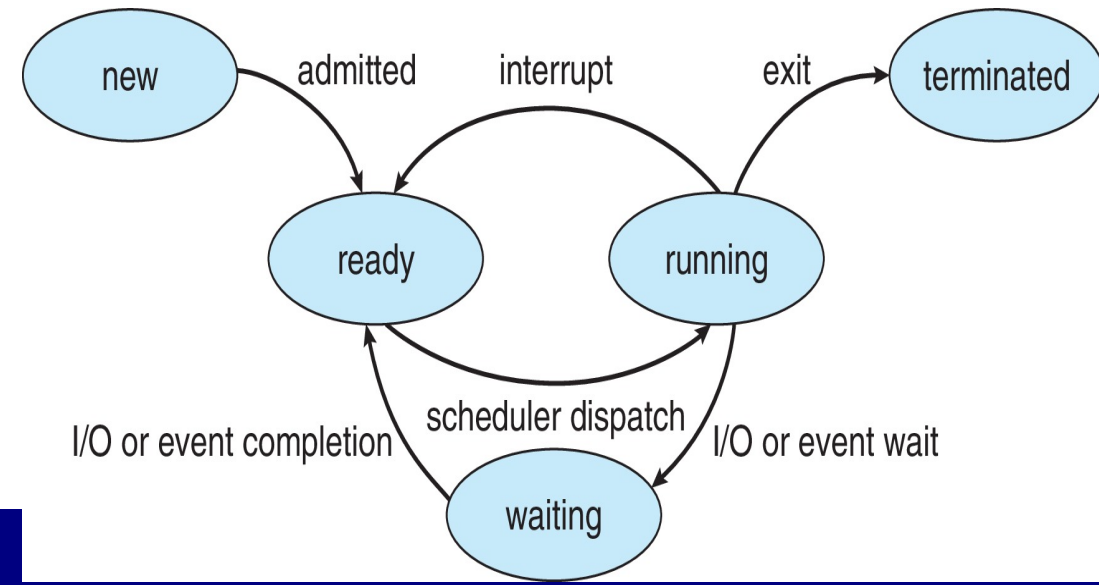
**New:** The process is being created

**Running:** Instructions are being executed

**Waiting:** The process is **waiting for some I/O event** (e.g., keyboard input, file read).

**Ready:** The process is waiting to be assigned to a processor

**Terminated:** The process has finished execution



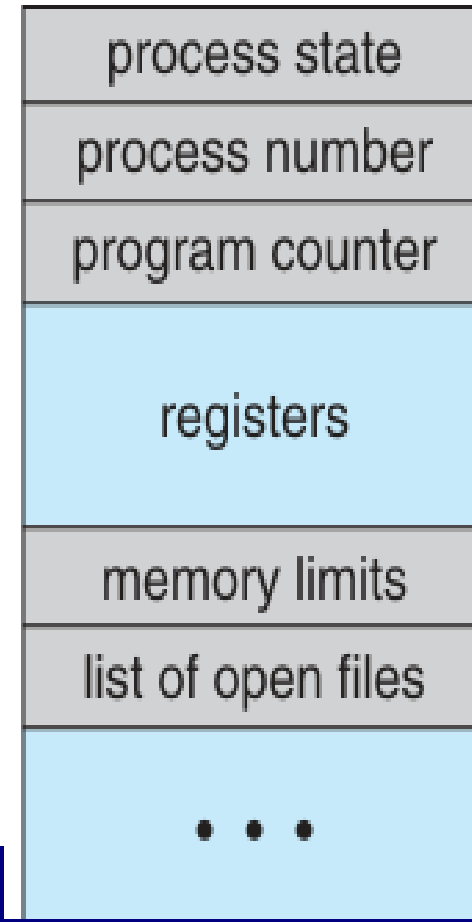
- The **Process Control Block (PCB)** is a **data structure** used by the operating system to **store all information about a process**.
- Each process has its **own PCB**, which the OS uses to manage and schedule that process.
- It is also called task control block.

## Process State

- Describes the **current status** of the process.
- Common states: **New, Ready ,Running ,Waiting ,Terminated**

## Program Counter

- Contains the **address of the next instruction** to be executed for the process.



## **CPU Registers**

- Includes all processor-specific registers that the process is using: Accumulators, Index registers, Stack pointers, General-purpose registers

## **CPU-scheduling information**

- This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters.

## **Memory Management Information**

- May include: Base and limit registers, Page tables, Segment tables

## **Accounting information**

- It includes the amount of CPU and real time used, time limits and so on.

## **I/O status information**

- It includes the list of I/O devices allocated to the process, a list of open files, and so on.



- CPU or process scheduling is the process by which the operating system selects the next process to be executed on the CPU

## Key components in Process Scheduling

- **Scheduling Queues**

- Job Queue
- Ready Queue
- Device (I/O) Queue

- **Schedulers**

- Long-Term Scheduler
- Short-Term Scheduler
- Medium-Term Scheduler

- **Context Switch**

## Job Queue

- Contains **all processes** that enter the system.
- As soon as a process is created, it is put into this queue.

## Ready Queue

- The processes that are residing in main memory and are ready and waiting to execute are kept in ready queue.

## Device Queue

- The list of processes waiting for a particular I/O device is called a device queue.
- Each I/O device (like disk, printer, keyboard) has its own separate device queue.

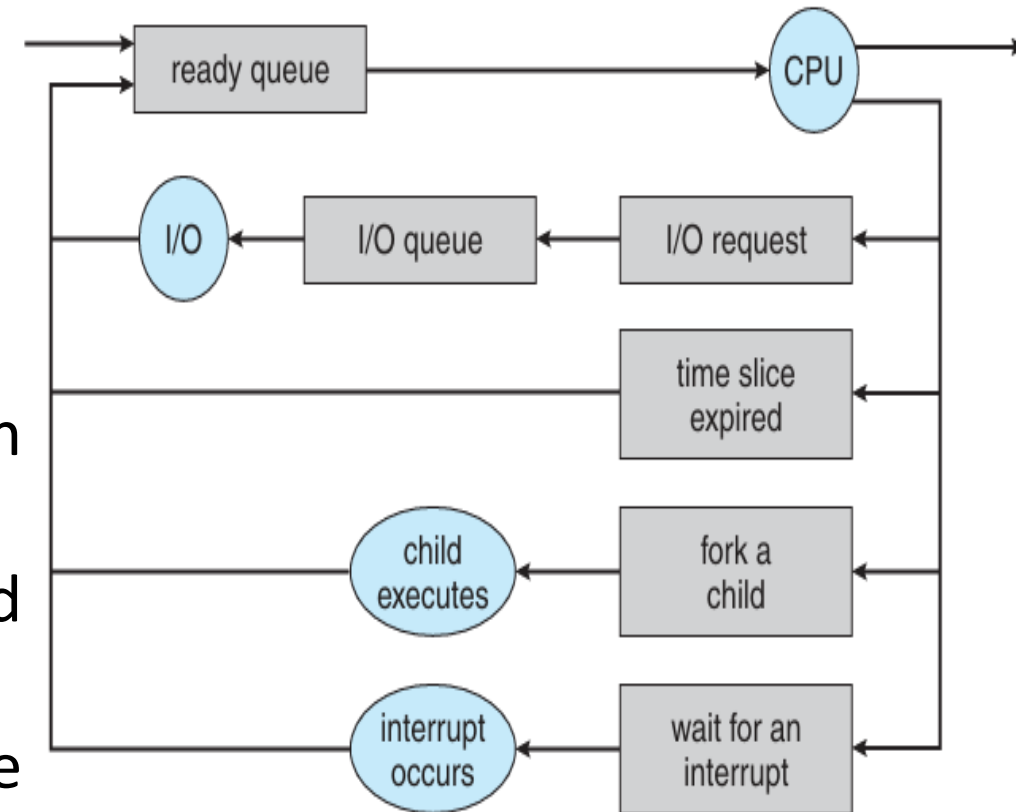
# Queueing-diagram representation of process scheduling

- It shows how processes move between different queues and components of the CPU scheduling system.

A new process is initially put in the ready queue.

**While process is executing on the CPU, one of several events could occur:**

- The process could issue an I/O request and then be placed in an I/O queue.
- The process could create a new child process and wait for the child's termination.
- The process could be removed forcibly from the CPU, as a result of an interrupt, and be put back in the ready queue.



## Long-Term Scheduler (Job Scheduler)

- Selects processes from the **job pool** (newly created processes) and loads them into **main memory** (ready queue).
- Runs **less frequently**.
- Controls the **degree of multiprogramming** (number of processes in memory).

## Short-Term Scheduler (CPU Scheduler)

- The short-term scheduler selects one of the processes that are in the ready queue and assigns the CPU to that process for execution
- Runs **very frequently**
- Very fast

## Medium-term scheduler

- A **medium-term scheduler swaps out processes** from main memory to secondary storage to reduce memory load, and **swaps them back in** when sufficient memory becomes available, allowing the process to resume execution.

## Context Switch

- A **context switch** happens when the CPU switches from one process to another.

Steps involved:

- Save** the context of the currently running process into its PCB.
- Load** the context of the next scheduled process from its PCB.
- When an **interrupt** occurs (e.g., due to I/O, timer, or system call), the operating system must **pause** the currently running process.
- To do this safely, it needs to **save the current state (context)** of that process so it can resume it later from the exact same point.

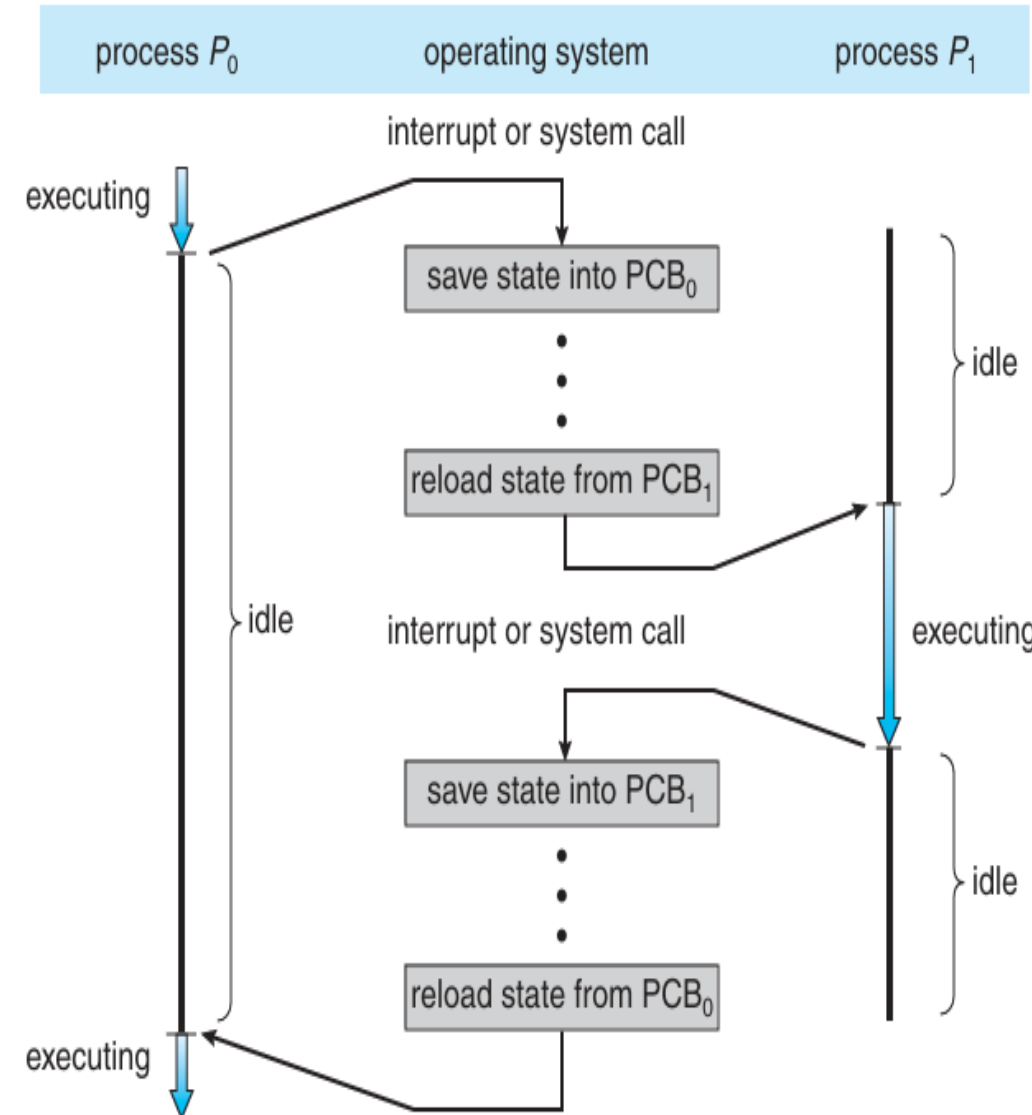


Figure 3.4 Diagram showing CPU switch from process to process.

- CPU scheduling algorithms are evaluated based on several **performance criteria**.
- These determine how efficiently the CPU and system resources are used.

## CPU utilization

- We want to keep the CPU as busy as possible.
- Conceptually, CPU utilization can range from 0 to 100 percent.
- In a real system, it should range from 40 percent (for a lightly loaded system) to 90 percent (for a heavily loaded system).

## Throughput

- Number of processes that are completed per time unit.
- For long processes ,this rate may be one process per hour; for short ,it may be ten processes per second

## Turnaround Time

- Total time taken from process submission to completion.
- Turnaround time is the sum of the periods spent waiting to get into memory, waiting in the ready queue, executing on the CPU, and doing I/O

## Waiting time

- Waiting time is the sum of the periods spent waiting in the ready queue.

## Response Time

- Time from process submission to the **first response** (not completion).
- Response time, is the time it takes to start responding, not the time it takes to output the response.
- It is desirable to maximize CPU utilization and throughput and to minimize turnaround time, waiting time, and response time.

## Preemptive Scheduling

- Preemptive Scheduling allows the operating system to interrupt a currently running process and allocate the CPU to another process.

### When Preemption Occurs:

- A **higher-priority** process arrives.
- The current process **exceeds its time slice** (e.g., in Round Robin).
- A process with **shorter remaining time** becomes ready (e.g., in SRTF).

**Ex:** Round Robin (RR), Shortest Remaining Time First (SRTF), Preemptive Priority Scheduling

## Non-Preemptive Scheduling

- Non-Preemptive Scheduling is a CPU scheduling method where once a process starts executing, it runs to completion or until it voluntarily releases the CPU (e.g., by requesting I/O or terminating).

**Ex:** First-Come, First-Served (FCFS), Non-Preemptive Shortest Job First (SJF), Non-Preemptive Priority Scheduling



## Non-Preemptive Scheduling Algorithms:

- First-Come, First-Served (FCFS)
- Shortest Job First (SJF) – Non-Preemptive
- Priority Scheduling – Non-Preemptive

## Preemptive Scheduling Algorithms:

- Shortest Remaining Time First (SRTF) (*Preemptive version of SJF*)
- Priority Scheduling – Preemptive
- Round Robin (RR)
- Multilevel Queue Scheduling
- Multilevel Feedback Queue Scheduling

# Formulas

## Turnaround Time (TAT):

$TAT = \text{Completed Time} - \text{Arrival Time (AT)}$

$TAT = CT - AT$

## Waiting Time (WT):

$WT = TAT - \text{Burst Time}$

$= TAT - BT$

## Response Time(RT):

$\text{Response Time} = \text{Start Time} - \text{Arrival Time}$

# First-Come, First-Served (FCFS)

- First-Come, First-Served (FCFS) is the **simplest CPU scheduling algorithm**.
- In FCFS, the process that arrives **first** in the ready queue is executed **first**, and so on.
- The **CPU is assigned** to the process in the order of **arrival time**.
- Once a process starts executing, it **runs to completion**.

# First-Come, First-Served (FCFS)

Consider the following set of processes with their respective **burst times**. All processes arrive at **time 0**.

| Process | Burst Time |
|---------|------------|
| P1      | 24         |
| P2      | 3          |
| P3      | 3          |

The processes are scheduled using the **First-Come, First-Served (FCFS)** scheduling algorithm in the order:  
**P1 → P2 → P3**

## Tasks:

1. Draw the **Gantt chart** representing the process execution.
2. Calculate the following for each process:
  - Completion Time (CT)
  - Turnaround Time (TAT)
  - Waiting Time (WT)
  - Response Time (RT)
3. Calculate the **average turnaround time**, **average waiting time**, and **average response time**.

# First-Come, First-Served (FCFS)

## Gantt Chart :



| Process | AT | BT | CT | TAT = CT - AT | WT = TAT - BT | RT = Start Time - AT |
|---------|----|----|----|---------------|---------------|----------------------|
| P1      | 0  | 24 | 24 | 24            | 0             | 0                    |
| P2      | 0  | 3  | 27 | 27            | 24            | 24                   |
| P3      | 0  | 3  | 30 | 30            | 27            | 27                   |

- Average Turnaround Time (TAT):

$$\frac{24 + 27 + 30}{3} = \frac{81}{3} = 27$$

- Average Waiting Time (WT):

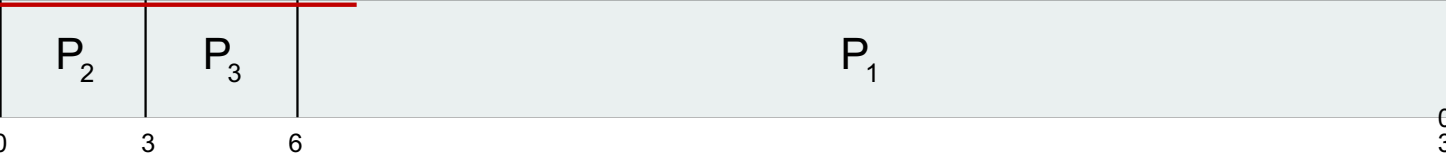
$$\frac{0 + 24 + 27}{3} = \frac{51}{3} = 17$$

- Average Response Time (RT):

$$\frac{0 + 24 + 27}{3} = \frac{51}{3} = 17$$

Suppose that the processes arrive in the order:  $P_2$  ,  $P_3$  ,  $P_1$

Gantt Chart :



| Process | Arrival Time (AT) | Burst Time (BT) | Completion Time (CT) | Turnaround Time (TAT = CT - AT) | Waiting Time (WT = TAT - BT) | Response Time (RT = Start - AT) |
|---------|-------------------|-----------------|----------------------|---------------------------------|------------------------------|---------------------------------|
| P2      | 0                 | 3               | 3                    | 3                               | 0                            | 0                               |
| P3      | 0                 | 3               | 6                    | 6                               | 3                            | 3                               |
| P1      | 0                 | 24              | 30                   | 30                              | 6                            | 6                               |

• Average Turnaround Time:

$$\frac{3 + 6 + 30}{3} = \frac{39}{3} = 13$$

• Average Waiting Time:

$$\frac{0 + 3 + 6}{3} = \frac{9}{3} = 3$$

• Average Response Time:

$$0 + 3 + 6 = 9 = 3$$

## Problem-3

Consider the following set of processes, each with their **arrival time** and **burst time**:

| Process | Arrival Time (AT) | Burst Time (BT) |
|---------|-------------------|-----------------|
| P1      | 0                 | 5               |
| P2      | 1                 | 3               |
| P3      | 2                 | 8               |
| P4      | 3                 | 6               |

- Construct the **Gantt chart**.
- Calculate the following for each process:
  - **Completion Time (CT)**
  - **Turnaround Time (TAT)**
  - **Waiting Time (WT)**
  - **Response Time (RT)**
- Compute the **average turnaround time**, **average waiting time**, and **average response time**.

## Problem-3

### Gantt Chart :

|   |    |   |    |   |    |    |    |    |
|---|----|---|----|---|----|----|----|----|
|   | P1 |   | P2 |   | P3 |    | P4 |    |
| 0 |    | 5 |    | 8 |    | 16 |    | 22 |

| Process | AT | BT | Start Time | CT | TAT = CT - AT | WT = TAT - BT | RT = Start - AT |
|---------|----|----|------------|----|---------------|---------------|-----------------|
| P1      | 0  | 5  | 0          | 5  | 5             | 0             | 0               |
| P2      | 1  | 3  | 5          | 8  | 7             | 4             | 4               |
| P3      | 2  | 8  | 8          | 16 | 14            | 6             | 6               |
| P4      | 3  | 6  | 16         | 22 | 19            | 13            | 13              |

- Average Turnaround Time (TAT):

$$\frac{5 + 7 + 14 + 19}{4} = \frac{45}{4} = 11.25$$

- Average Waiting Time (WT):

$$\frac{0 + 4 + 6 + 13}{4} = \frac{23}{4} = 5.75$$

- Average Response Time (RT):

$$\frac{0 + 4 + 6 + 13}{4} = \frac{23}{4} = 5.75$$



- when a long (CPU-bound) process occupies the CPU, and all other shorter processes must wait for it to complete.
- This effect results in **lower CPU and device utilization**.
- **If shorter processes were allowed to execute first**, overall performance, responsiveness, and resource utilization would be significantly improved.

# Shortest Job First (SJF)/SJN

- This algorithm associates with each process the length of the process's next CPU burst.
- Shortest Job First (SJF), also called **Shortest Job Next (SJN)**, is a **non-preemptive** scheduling algorithm in which the process with the **shortest burst time (execution time)** is selected next for execution.
- When the CPU is available, it is assigned to the process that has the smallest next CPU burst.
- If the next CPU bursts of two processes are the same, FCFS scheduling is used to break the tie.

# Shortest Job First (SJF)/SJN

Consider the following set of processes, with their respective burst times. All processes arrive at **time 0**.

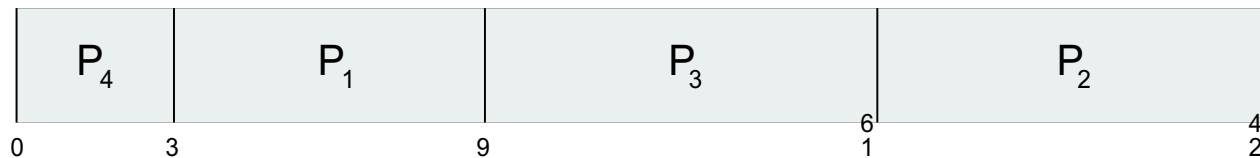
| Process | Burst Time |
|---------|------------|
| P1      | 6          |
| P2      | 8          |
| P3      | 7          |
| P4      | 3          |

Using the **Shortest Job First (SJF) Non-Preemptive Scheduling Algorithm**, do the following:

1. Draw the **Gantt Chart**.
2. Calculate the following for each process:
  - **Completion Time (CT)**
  - **Turnaround Time (TAT) = CT - Arrival Time**
  - **Waiting Time (WT) = TAT - Burst Time**
  - **Response Time (RT) = Time when process starts - Arrival Time**
3. Compute the **Average Turnaround Time**, **Average Waiting Time**, and **Average Response Time**.

## Gantt Chart :

# Shortest Job First (SJF)/SJN



| Process | Burst Time (BT) | Completion Time (CT) | Turnaround Time (TAT = CT - AT) | Waiting Time (WT = TAT - BT) | Response Time (RT = Start - AT) |
|---------|-----------------|----------------------|---------------------------------|------------------------------|---------------------------------|
| P4      | 3               | 3                    | 3                               | 0                            | 0                               |
| P1      | 6               | 9                    | 9                               | 3                            | 3                               |
| P3      | 7               | 16                   | 16                              | 9                            | 9                               |
| P2      | 8               | 24                   | 24                              | 16                           | 16                              |

**Response Time** = Time when process **first starts** executing - **Arrival Time**

### Step 4: Averages

- Average Turnaround Time (TAT) =  $(3 + 9 + 16 + 24) / 4 = 13$
- Average Waiting Time (WT) =  $(0 + 3 + 9 + 16) / 4 = 7$
- Average Response Time (RT) =  $(0 + 3 + 9 + 16) / 4 = 7$

# Shortest Job First (SJF)/SJN

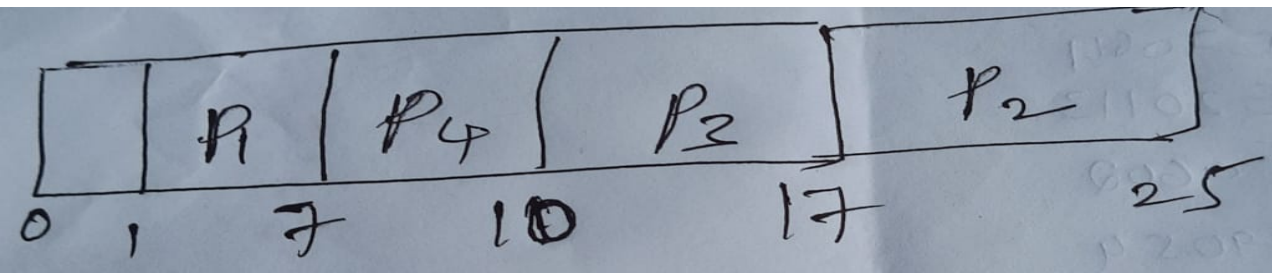
Consider the following set of processes with their respective **arrival times** and **burst times**:

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| P1      | 1            | 6          |
| P2      | 2            | 8          |
| P3      | 3            | 7          |
| P4      | 4            | 3          |

Using the **Shortest Job First (SJF) Non-Preemptive Scheduling Algorithm**, do the following:

1. Draw the **Gantt Chart**.
2. Calculate the following for each process:
  - **Completion Time (CT)**
  - **Turnaround Time (TAT) = CT - Arrival Time**
  - **Waiting Time (WT) = TAT - Burst Time**
  - **Response Time (RT) = Time when process starts - Arrival Time**
3. Compute the **Average Turnaround Time**, **Average Waiting Time**, and **Average Response Time**.

# Gantt Chart :



## 🕒 Step-by-Step Scheduling & Gantt Chart Construction

- At time 1, only P1 has arrived. So:
  - P1 starts at 1, ends at 7 (1 + 6)
- At time 7, all remaining processes (P2, P3, P4) have arrived.
  - Ready queue: P2 (8), P3 (7), P4 (3)
  - P4 has the shortest burst time → Starts at 7, ends at 10
- At time 10, remaining: P2 (8), P3 (7)
  - P3 is shorter → Starts at 10, ends at 17
- P2 is the last → Starts at 17, ends at 25

| Process | AT | BT | Start | CT | TAT = CT – AT | WT = TAT – BT | RT = Start – AT |
|---------|----|----|-------|----|---------------|---------------|-----------------|
| P1      | 1  | 6  | 1     | 7  | 6             | 0             | 0               |
| P2      | 2  | 8  | 17    | 25 | 23            | 15            | 15              |
| P3      | 3  | 7  | 10    | 17 | 14            | 7             | 7               |
| P4      | 4  | 3  | 7     | 10 | 6             | 3             | 3               |

### 📌 Average Times:

- Average Turnaround Time (TAT):

$$\frac{6+23+14+6}{4} = \frac{49}{4} = 12.25$$

- Average Waiting Time (WT):

$$\frac{0+15+7+3}{4} = \frac{25}{4} = 6.25$$

- Average Response Time (RT):

$$\frac{0+15+7+3}{4} = \frac{25}{4} = 6.25$$

# shortest-remaining-time-first



- The SJF algorithm can be either preemptive or non preemptive.
- The choice arises when a new process arrives at the ready queue while a previous process is still executing.
- The next CPU burst of the newly arrived process may be shorter than remaining time of currently executing process.
- A preemptive SJF algorithm will preempt the currently executing process, whereas a non preemptive SJF algorithm will allow the currently running process to finish its CPU burst. Preemptive SJF scheduling is sometimes called shortest-remaining-time-first scheduling

# shortest-remaining-time-first

Consider the following set of processes, with their arrival times and burst times:

| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| P1      | 0            | 8          |
| P2      | 1            | 4          |
| P3      | 2            | 9          |
| P4      | 3            | 5          |

Using the **Shortest Remaining Time First (SRTF)** scheduling algorithm (preemptive version of SJF), perform the following tasks:

(a) Construct the Gantt Chart showing the order of execution over time.

(b) For each process, calculate:

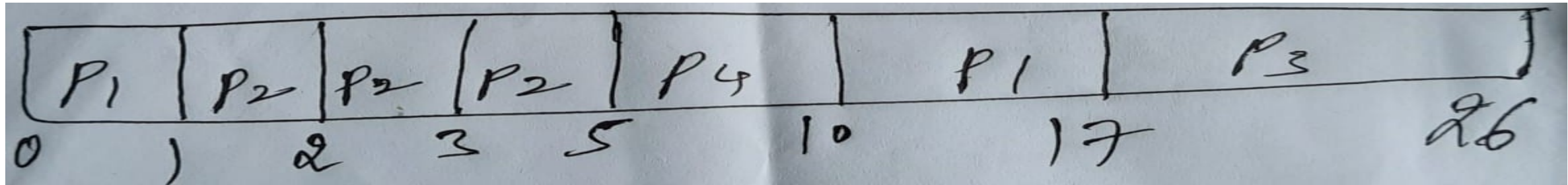
- Completion Time (CT)
- Turnaround Time ( $TAT = CT - \text{Arrival Time}$ )
- Waiting Time ( $WT = TAT - \text{Burst Time}$ )
- Response Time ( $RT = \text{First Start Time} - \text{Arrival Time}$ )

(c) Calculate:

- Average Turnaround Time
- Average Waiting Time
- Average Response Time







## ◆ Explanation of Gantt Chart:

1. At time 0: Only P1 is available → P1 starts execution.
2. At time 1: P2 arrives with shorter BT (4) → P2 preempts P1.
3. At time 2: P3 arrives, but P2 has lesser BT → Continue P2.
4. At time 3: P4 arrives, but still P2 has shortest remaining → Continue P2.
5. P2 finishes at time 5.
6. Among remaining, P4 has shortest BT → Runs from 5 to 10.
7. Next, P1 resumes and runs from 10 to 16.
8. Finally, P3 runs from 16 to 25.



| Process | Arrival | Burst | Start Time | Completion | Turnaround (CT - AT) | Waiting (TAT - BT) | Response (Start - AT) |
|---------|---------|-------|------------|------------|----------------------|--------------------|-----------------------|
| P1      | 0       | 8     | 0          | 17         | 17                   | 9                  | 0                     |
| P2      | 1       | 4     | 1          | 5          | 4                    | 0                  | 0                     |
| P3      | 2       | 9     | 17         | 26         | 24                   | 15                 | 15                    |
| P4      | 3       | 5     | 5          | 10         | 7                    | 2                  | 2                     |

 Average Times:

- Average Turnaround Time =  $(17 + 4 + 24 + 7) / 4 = 13$
- Average Waiting Time =  $(9 + 0 + 15 + 2) / 4 = 6.5$
- Average Response Time =  $(0 + 0 + 15 + 2) / 4 = 4.25$

# shortest-remaining-time-first



| Process | Arrival Time | Burst Time |
|---------|--------------|------------|
| P1      | 0            | 6          |
| P2      | 2            | 2          |
| P3      | 4            | 1          |
| P4      | 5            | 3          |

Consider the following set of processes with their respective **arrival times** and **burst times**. Using **Shortest Remaining Time First (SRTF)** scheduling, calculate:

- Completion Time (CT)
- Turnaround Time (TAT)
- Waiting Time (WT)
- Draw the Gantt chart.



## Gantt Chart:

|         |    |    |    |    |
|---------|----|----|----|----|
| P1      | P2 | P3 | P4 | P1 |
| 0     2 | 4  | 5  | 8  | 11 |

- Time 0–2: P1 runs
- Time 2–4: P2 (shorter job) preempts P1
- Time 4–5: P3 arrives, preempts and completes
- Time 5–8: P4 runs
- Time 8–11: P1 resumes and completes

- ◆ Average Turnaround Time =  $(11 + 2 + 1 + 3) / 4 = 4.25$
- ◆ Average Waiting Time =  $(5 + 0 + 0 + 0) / 4 = 1.25$
- ◆ Average Response Time =  $(0 + 0 + 0 + 0) / 4 = 0.00$

| Process | Arrival Time | Burst Time | Start Time | Completion Time | Turnaround Time (TAT) | Waiting Time (WT) | Response Time (RT) |
|---------|--------------|------------|------------|-----------------|-----------------------|-------------------|--------------------|
| P1      | 0            | 6          | 0          | 11              | 11                    | 5                 | 0                  |
| P2      | 2            | 2          | 2          | 4               | 2                     | 0                 | 0                  |
| P3      | 4            | 1          | 4          | 5               | 1                     | 0                 | 0                  |
| P4      | 5            | 3          | 5          | 8               | 3                     | 0                 | 0                  |

# Priority Scheduling Algorithm



- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer  $\equiv$  highest priority)
- Equal priority processes follow FCFS.
- Can be preemptive or non-preemptive.

# Priority Scheduling Algorithm

Consider the following set of processes, where lower priority number means higher priority. All processes arrive at time 0.

Using **Non-Preemptive Priority Scheduling**, determine the **completion time**, **turnaround time**, and **waiting time** for each process. Also, compute the **average turnaround time** and **average waiting time**.

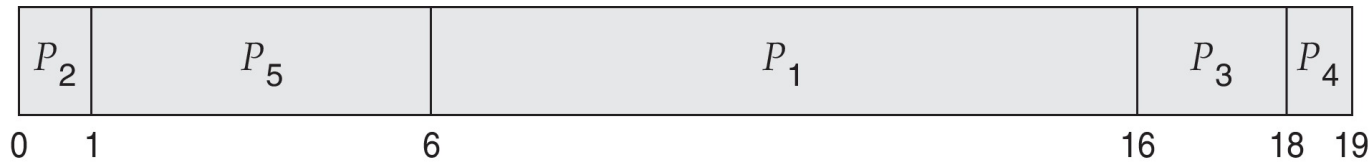
| Process | Burst Time (ms) | Priority |
|---------|-----------------|----------|
| P1      | 10              | 3        |
| P2      | 1               | 1        |
| P3      | 2               | 4        |
| P4      | 1               | 5        |
| P5      | 5               | 2        |

Draw the **Gantt chart** for the schedule.

# Non-Preemptive Priority Scheduling



## Gantt Chart



| Process | Burst | Priority | Completion | Turnaround (TAT) | Waiting |
|---------|-------|----------|------------|------------------|---------|
| P1      | 10    | 3        | 16         | 16               | 6       |
| P2      | 1     | 1        | 1          | 1                | 0       |
| P3      | 2     | 4        | 18         | 18               | 16      |
| P4      | 1     | 5        | 19         | 19               | 18      |
| P5      | 5     | 2        | 6          | 6                | 1       |

## Averages

- Sum of Turnaround = 60 → Average Turnaround =  $60 / 5 = 12.0$
- Sum of Waiting = 41 → Average Waiting =  $41 / 5 = 8.2$

# Preemptive Priority Scheduling



**Rule (preemptive):** at any time run the ready process with the smallest priority number; new higher-priority arrivals preempt the running process.

Consider the following set of processes, where **lower priority number indicates higher priority**. The scheduling is done using **Preemptive Priority Scheduling**.

| Process | Arrival Time (ms) | Burst Time (ms) | Priority |
|---------|-------------------|-----------------|----------|
| P1      | 0                 | 8               | 2        |
| P2      | 1                 | 4               | 1        |
| P3      | 2                 | 9               | 3        |
| P4      | 3                 | 5               | 2        |

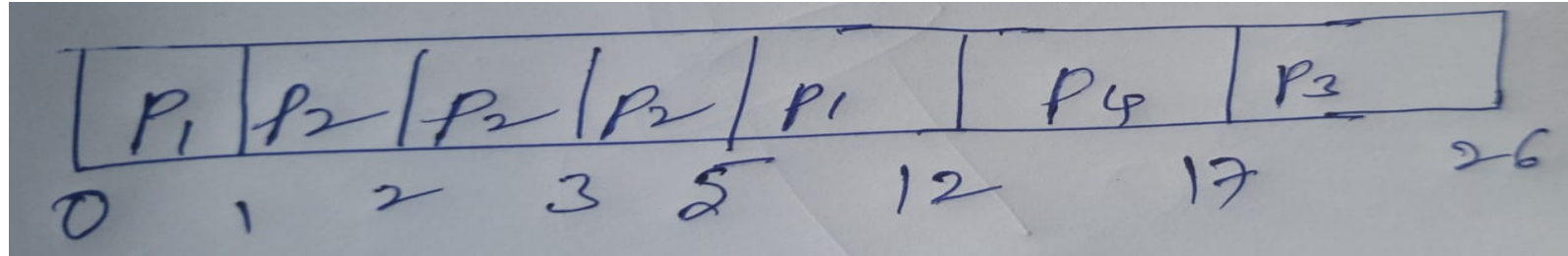
1. Draw the **Gantt chart** for the schedule.
2. Calculate the following for each process:
  - Completion Time (CT)
  - Turnaround Time (TAT)
  - Waiting Time (WT)
  - Response Time (RT)
3. Compute the **average TAT**, **average WT**, and **average RT**.



# Preemptive Priority Scheduling



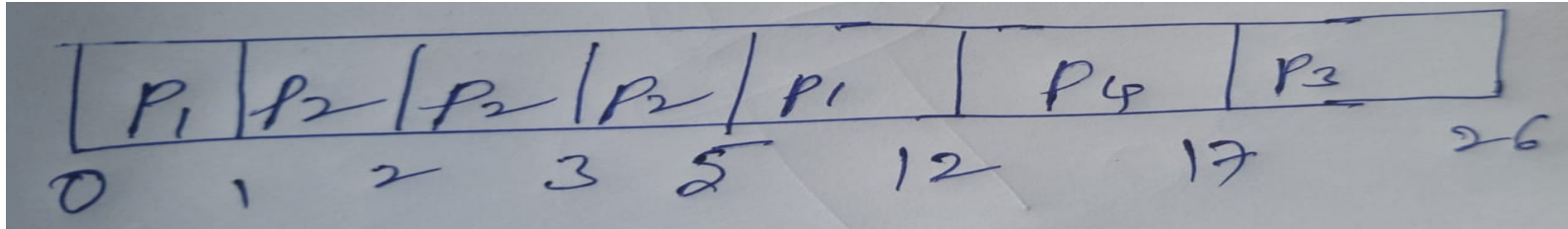
## Gantt Chart



| Time | Event                                                                                                        |
|------|--------------------------------------------------------------------------------------------------------------|
| t=0  | P1 arrives → CPU runs <b>P1</b> (priority 2).                                                                |
| t=1  | P2 arrives (priority 1 — higher than P1's priority 2) → <b>P1 is preempted</b> → CPU switches to <b>P2</b> . |
| t=2  | P3 arrives (priority 3 — lower than P2's priority 1) → <b>No preemption</b> , P2 continues.                  |
| t=3  | P4 arrives (priority 2 — still lower than P2's priority 1) → <b>No preemption</b> , P2 continues.            |
| t=5  | P2 finishes → CPU picks next highest priority process (P1, priority 2 and FCFS) and runs it.                 |

# Preemptive Priority Scheduling

## Gantt Chart



| Process | AT | BT | Priority | ST | CT | TAT = CT-AT   | WT =<br>TAT-Burst | RT = ST-AT    |
|---------|----|----|----------|----|----|---------------|-------------------|---------------|
| P1      | 0  | 8  | 2        | 0  | 12 | $12 - 0 = 12$ | $12 - 8 = 4$      | $0 - 0 = 0$   |
| P2      | 1  | 4  | 1        | 1  | 5  | $5 - 1 = 4$   | $4 - 4 = 0$       | $1 - 1 = 0$   |
| P3      | 2  | 9  | 3        | 17 | 26 | $26 - 2 = 24$ | $24 - 9 = 15$     | $17 - 2 = 15$ |
| P4      | 3  | 5  | 2        | 12 | 17 | $17 - 3 = 14$ | $14 - 5 = 9$      | $12 - 3 = 9$  |

- Average Turnaround Time (TAT) =  $(12 + 4 + 24 + 14) / 4 = \mathbf{13.5}$
- Average Waiting Time (WT) =  $(4 + 0 + 15 + 9) / 4 = \mathbf{7.0}$
- Average Response Time (RT) =  $(0 + 0 + 15 + 9) / 4 = \mathbf{6.0}$

# Preemptive Priority Scheduling

| Process | Arrival Time (ms) | Burst Time (ms) | Priority |
|---------|-------------------|-----------------|----------|
| P1      | 0                 | 6               | 3        |
| P2      | 0                 | 4               | 1        |
| P3      | 5                 | 8               | 4        |
| P4      | 5                 | 3               | 2        |
| P5      | 2                 | 5               | 1        |

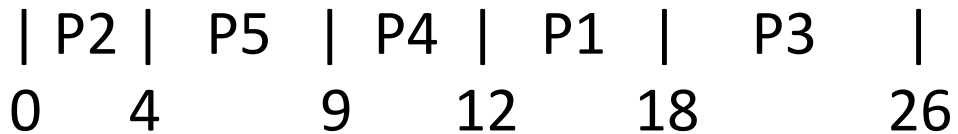
## Tasks:

1. Draw the **Gantt chart** for each case.
2. For each process, calculate:
  - Completion Time (CT)
  - Turnaround Time (TAT) = CT – AT
  - Waiting Time (WT) = TAT – BT
  - Response Time (RT) = First Start Time – AT
3. Compute **Average TAT**, **Average WT**, and **Average RT** for each case.

# Preemptive Priority Scheduling



## Gantt Chart



| Process | AT | BT | PR | ST | CT | TAT = CT-AT   | WT = TAT-BT   | RT = ST- AT   |
|---------|----|----|----|----|----|---------------|---------------|---------------|
| P1      | 0  | 6  | 3  | 12 | 18 | $18 - 0 = 18$ | $18 - 6 = 12$ | $12 - 0 = 12$ |
| P2      | 0  | 4  | 1  | 0  | 4  | $4 - 0 = 4$   | $4 - 4 = 0$   | $0 - 0 = 0$   |
| P3      | 5  | 8  | 4  | 18 | 26 | $26 - 5 = 21$ | $21 - 8 = 13$ | $18 - 5 = 13$ |
| P4      | 5  | 3  | 2  | 9  | 12 | $12 - 5 = 7$  | $7 - 3 = 4$   | $9 - 5 = 4$   |
| P5      | 2  | 5  | 1  | 4  | 9  | $9 - 2 = 7$   | $7 - 5 = 2$   | $4 - 2 = 2$   |

- Average Turnaround Time (TAT) =  $(18 + 4 + 21 + 7 + 7) / 5 = 11.4$
- Average Waiting Time (WT) =  $(12 + 0 + 13 + 4 + 2) / 5 = 6.2$
- Average Response Time (RT) =  $(12 + 0 + 13 + 4 + 2) / 5 = 6.2$

## Problem

### Starvation (Indefinite Blocking):

- low priority processes may never execute
- Low priority processes may suffer from **starvation** if higher priority processes keep arriving continuously, causing the low priority ones to never get executed.

### Solution

#### Aging

- **Aging** is a technique used to **prevent starvation** in priority scheduling.
- It works by **gradually increasing the priority** of a process the longer it waits in the ready queue.
- For each unit of time a process waits, its priority is **increased**

# Round Robin (RR) Scheduling



- Round Robin (RR) is a **preemptive CPU scheduling algorithm** designed mainly for **time-sharing systems**.
- In RR, each process is assigned a fixed time unit called a **time quantum** or **time slice**. The CPU scheduler goes through the ready queue in a circular manner, allocating CPU to each process for one quantum at a time.

## How does RR work?

- All processes are placed in a **ready queue**.
- The CPU scheduler picks the first process from the ready queue and allocates the CPU for a time interval equal to the **time quantum (q)**.
- If the process completes within this time, it leaves the system.
- If the process does not complete, it is **preempted** after the time quantum expires and placed at the back of the ready queue.
- The CPU scheduler then picks the next process in the queue and repeats the cycle.
- This continues until all processes are finished.

# Round Robin (RR) Scheduling

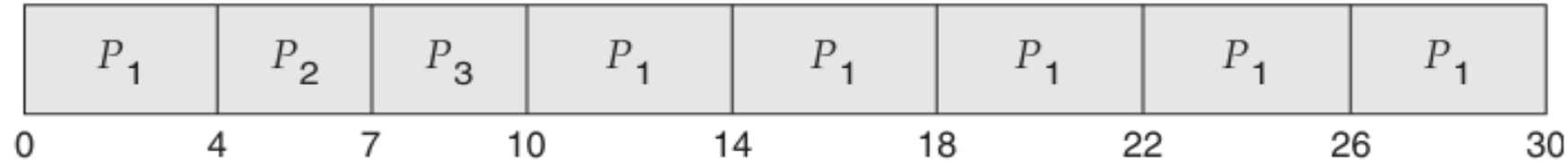
Consider the following set of processes that arrive at time 0. The CPU scheduling is done using the **Round Robin (RR)** algorithm with a time quantum of **4 ms**.

| Process | Burst Time (ms) |
|---------|-----------------|
| P1      | 24              |
| P2      | 3               |
| P3      | 3               |

## Tasks:

1. Draw the **Gantt chart** showing the execution order of the processes.
2. Calculate the **completion time**, **turnaround time**, and **waiting time** for each process.
3. Find the **average turnaround time** and **average waiting time**.

# Gantt Chart



| Process | B  | CT | TAT | WT |
|---------|----|----|-----|----|
| P1      | 24 | 30 | 30  | 6  |
| P2      | 3  | 7  | 7   | 4  |
| P3      | 3  | 10 | 10  | 7  |

- Average waiting time

$$\overline{WT} = \frac{\sum WT}{n} = \frac{6 + 4 + 7}{3} = \frac{17}{3} = \mathbf{5.67 \text{ ms}}$$

- Average turnaround time

$$\overline{TAT} = \frac{\sum TAT}{n} = \frac{30 + 7 + 10}{3} = \frac{47}{3} = \mathbf{15.67 \text{ ms}}$$



Given the following processes with their arrival times and burst times:

| Process Id | Arrival Time (ms) | Burst Time (ms) |
|------------|-------------------|-----------------|
| P1         | 0                 | 5               |
| P2         | 1                 | 3               |
| P3         | 2                 | 1               |
| P4         | 3                 | 2               |
| P5         | 4                 | 3               |

Using **Round Robin (RR)** scheduling algorithm with a **time quantum of 2 ms**, answer the following:

1. Draw the Gantt chart representing the process execution order.
2. Calculate the **Completion Time (CT)** for each process.
3. Calculate the **Turnaround Time (TAT)** for each process.
4. Calculate the **Waiting Time (WT)** for each process.
5. Calculate the **Response Time (RT)** for each process.

*(Response Time is the time from arrival to the first time the process gets the CPU)*

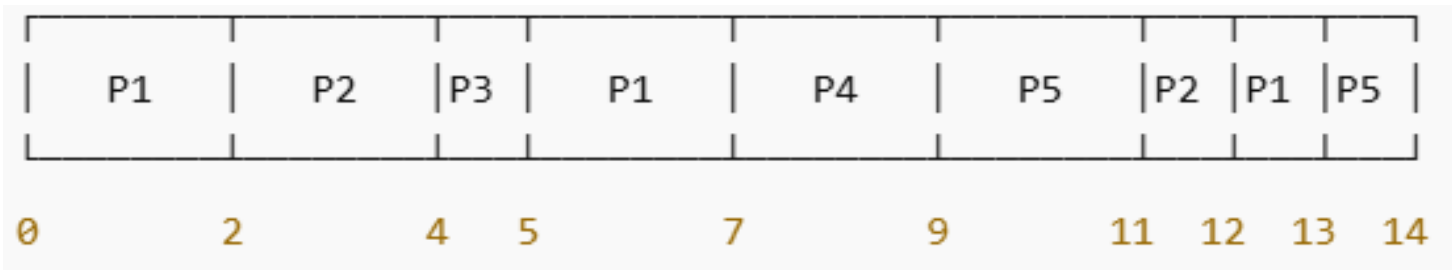
6. Calculate the average Turnaround Time, Waiting Time, and Response Time.

# Gantt Chart

Ready Queue: P1 P2 P3 P1 P4 P5 P2 P1 P5



**VIT-AP**  
UNIVERSITY



| Process | Arrival Time (AT) | Burst Time (BT) | Completion Time (CT) | Turnaround Time (TAT = CT - AT) | Waiting Time (WT = TAT - BT) | Response Time (RT = ST - AT) |
|---------|-------------------|-----------------|----------------------|---------------------------------|------------------------------|------------------------------|
| P1      | 0                 | 5               | 13                   | $13 - 0 = 13$                   | $13 - 5 = 8$                 | $0 - 0 = 0$                  |
| P2      | 1                 | 3               | 12                   | $12 - 1 = 11$                   | $11 - 3 = 8$                 | $2 - 1 = 1$                  |
| P3      | 2                 | 1               | 5                    | $5 - 2 = 3$                     | $3 - 1 = 2$                  | $4 - 2 = 2$                  |
| P4      | 3                 | 2               | 9                    | $9 - 3 = 6$                     | $6 - 2 = 4$                  | $7 - 3 = 4$                  |
| P5      | 4                 | 3               | 14                   | $14 - 4 = 10$                   | $10 - 3 = 7$                 | $9 - 4 = 5$                  |

- Average Turnaround Time =  $(13 + 11 + 3 + 6 + 10) / 5 = 43 / 5 = 8.6$
- Average Waiting Time =  $(8 + 8 + 2 + 4 + 7) / 5 = 29 / 5 = 5.8$
- Average Response Time =  $(0 + 1 + 2 + 4 + 5) / 5 = 12 / 5 = 2.4$

| Process Id | Arrival Time (ms) | Burst Time (ms) |
|------------|-------------------|-----------------|
| P1         | 0                 | 4               |
| P2         | 1                 | 5               |
| P3         | 3                 | 2               |
| P4         | 4                 | 1               |
| P5         | 6                 | 3               |

**Time Quantum (TQ) = 3 ms**

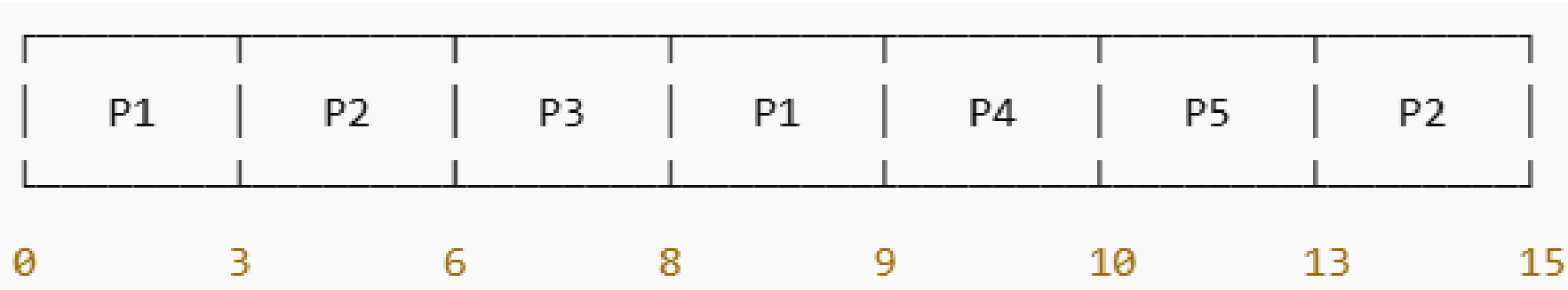
### Tasks:

1. Draw the Gantt chart for the process execution.
2. Find the Completion Time (CT) of each process.
3. Calculate Turnaround Time (TAT) for each process.
4. Calculate Waiting Time (WT) for each process.
5. Calculate Response Time (RT) for each process.
6. Compute average TAT, WT, and RT.



# Gantt Chart

Ready Queue: P1 P2 P3 P1 P4 P5 P2



| Process | AT | BT | CT | TAT = CT - AT | WT = TAT - BT | RT=ST- AT    |
|---------|----|----|----|---------------|---------------|--------------|
| P1      | 0  | 4  | 9  | $9 - 0 = 9$   | $9 - 4 = 5$   | $0 - 0 = 0$  |
| P2      | 1  | 5  | 15 | $15 - 1 = 14$ | $14 - 5 = 9$  | $3 - 1 = 2$  |
| P3      | 3  | 2  | 8  | $8 - 3 = 5$   | $5 - 2 = 3$   | $6 - 3 = 3$  |
| P4      | 4  | 1  | 10 | $10 - 4 = 6$  | $6 - 1 = 5$   | $9 - 4 = 5$  |
| P5      | 6  | 3  | 13 | $13 - 6 = 7$  | $7 - 3 = 4$   | $10 - 6 = 4$ |

- Avg Turnaround Time =  $(9 + 14 + 5 + 6 + 7) / 5 = 41 / 5 = \mathbf{8.2}$
- Avg Waiting Time =  $(5 + 9 + 3 + 5 + 4) / 5 = 26 / 5 = \mathbf{5.2}$
- Avg Response Time =  $(0 + 2 + 3 + 5 + 4) / 5 = 14 / 5 = \mathbf{2.8}$

# Round Robin (RR) Scheduling



## Advantages

**Fairness:** Each process gets an equal share of CPU time.

**Responsive:** Good for time-sharing systems, allowing interactive users to get CPU time regularly.

**No starvation:** Because of the cyclic order, every process will eventually get CPU time.

## Disadvantages

**Context Switching Overhead:** If time quantum is too small, frequent context switches happen, which can degrade performance.

**Choosing Quantum:** The time quantum value is critical — if it's too large, RR behaves like FCFS (First-Come-First-Served); if too small, excessive context switching occurs.

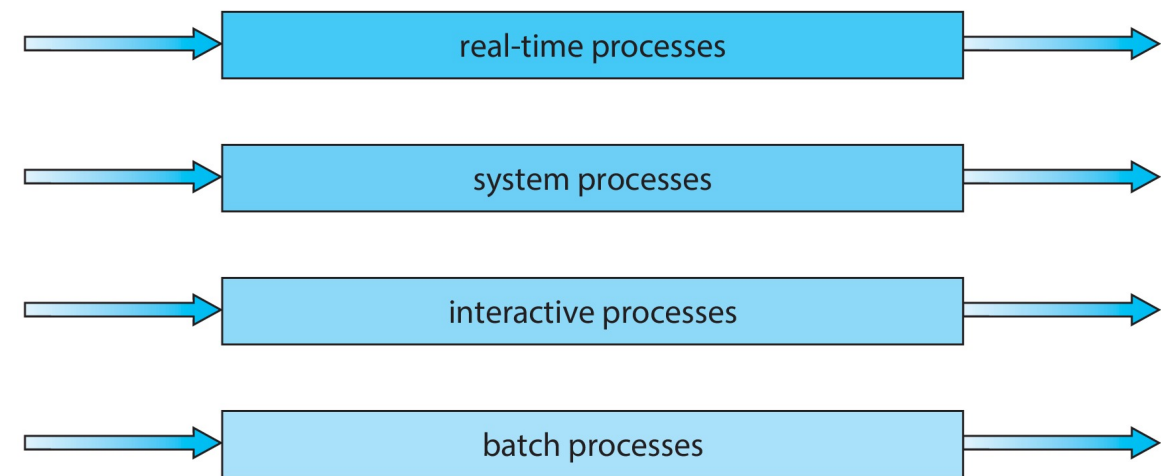
# Multilevel Queue Scheduling



- **Multilevel Queue Scheduling** is a CPU scheduling algorithm that partitions the ready queue into several separate queues.
- Each queue has its own **scheduling algorithm** and **priority level**.
- Processes are permanently assigned to **one queue** based on some characteristic like **process type**, **priority**, **memory** requirements, or process behavior.

| Process Type          | Scheduling Algorithm | Priority |
|-----------------------|----------------------|----------|
| System processes      | Round Robin          | Highest  |
| Interactive processes | Round Robin          | Medium   |
| Batch processes       | FCFS                 | Lowest   |

highest priority



lowest priority

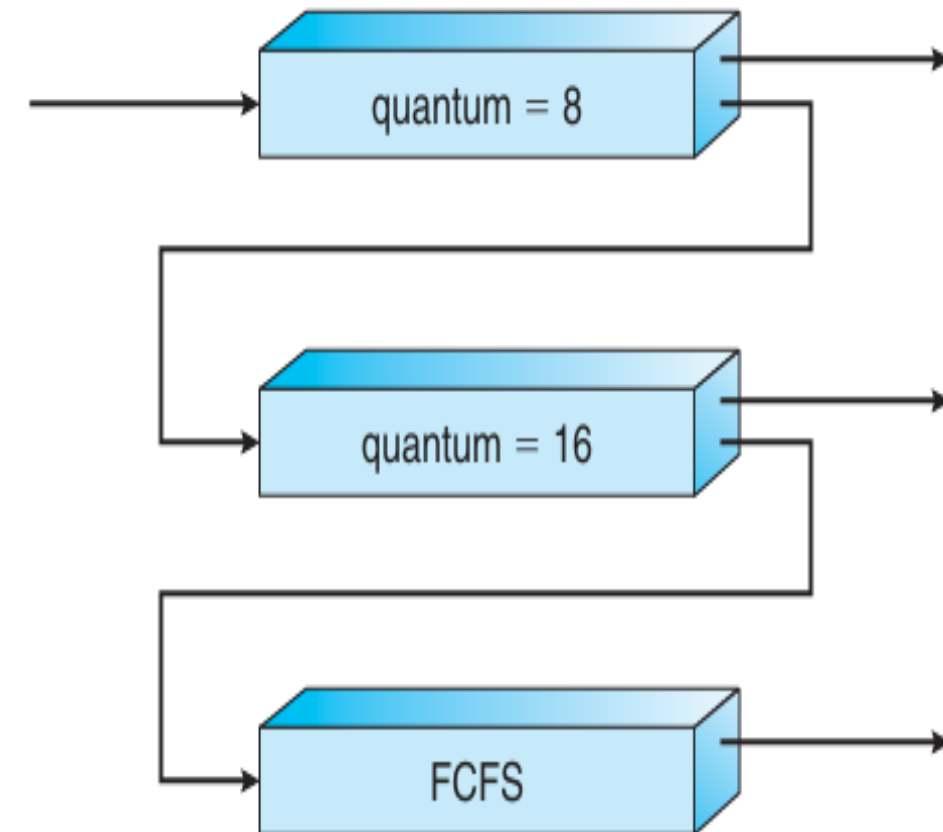
## How It Works

- The CPU scheduler always selects processes from the highest priority queue that is not empty.
- If that queue is empty, the scheduler looks for processes in the next lower priority queue.
- No process in the batch queue, could run unless the queues for system processes, and interactive processes were all empty.
- If an interactive process entered the ready queue while a batch process was running, the batch process would be preempted.

# Multilevel Feedback Queue Scheduling



- The multilevel feedback queue scheduling algorithm allows a process to move between queues.
- Consider a multilevel feedback queue scheduler with three queues, numbered from 0 to 2.
- The scheduler first executes all processes in queue 0.
- Only when queue 0 is empty will it execute processes in queue 1.
- Similarly, processes in queue 2 will be executed only if queues 0 and 1 are empty.
- A process that arrives for queue 1 will preempt a process in queue 2.
- A process in queue 1 will in turn be preempted by a process arriving for queue 0.

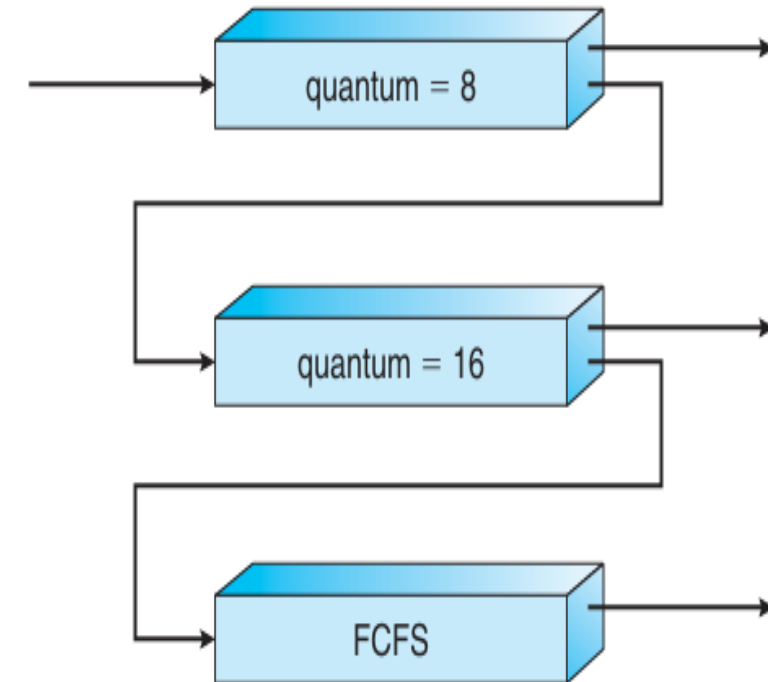




# Multilevel Feedback Queue Scheduling



- A process entering the ready queue is put in queue 0.
- A process in queue 0 is given a time quantum of 8 milliseconds.
- If it does not finish within this time, it is moved to the tail of queue 1.
- If queue 0 is empty, the process at the head of queue 1 is given a quantum of 16 milliseconds.
- If it does not complete, it is preempted and is put into queue 2.
- Processes in queue 2 are run on an FCFS basis but are run only when queues 0 and 1 are empty.



# Multilevel Feedback Queue Scheduling



- In general, a multilevel feedback queue scheduler is defined by the following parameters:
  - The number of queues
  - The scheduling algorithm for each queue
  - The method used to determine when to upgrade a process to a higher priority queue
  - The method used to determine when to demote a process to a lower priority queue

- Processes within a system may be *independent* or *cooperating*
- Any process that does not share data with any other process is **independent**.
- Any process that shares data with other processes is a **cooperating process**.
- **Inter process Communication (IPC)** refers to the mechanisms that allow processes to exchange data and coordinate their actions.

## Why IPC is Needed

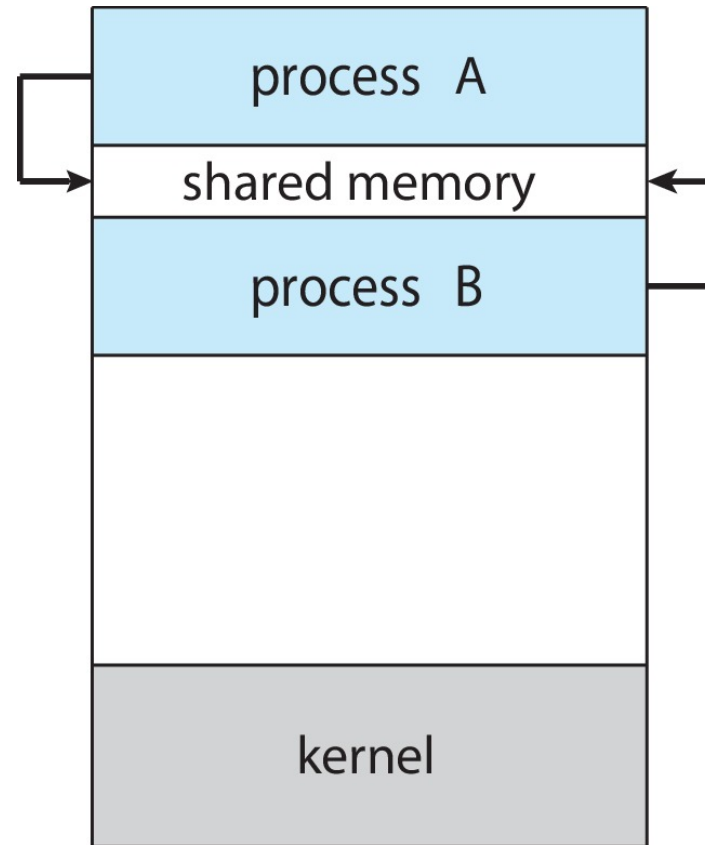
- Data Sharing
- Computation Speedup
- Modularity
- Convenience

# IPC models

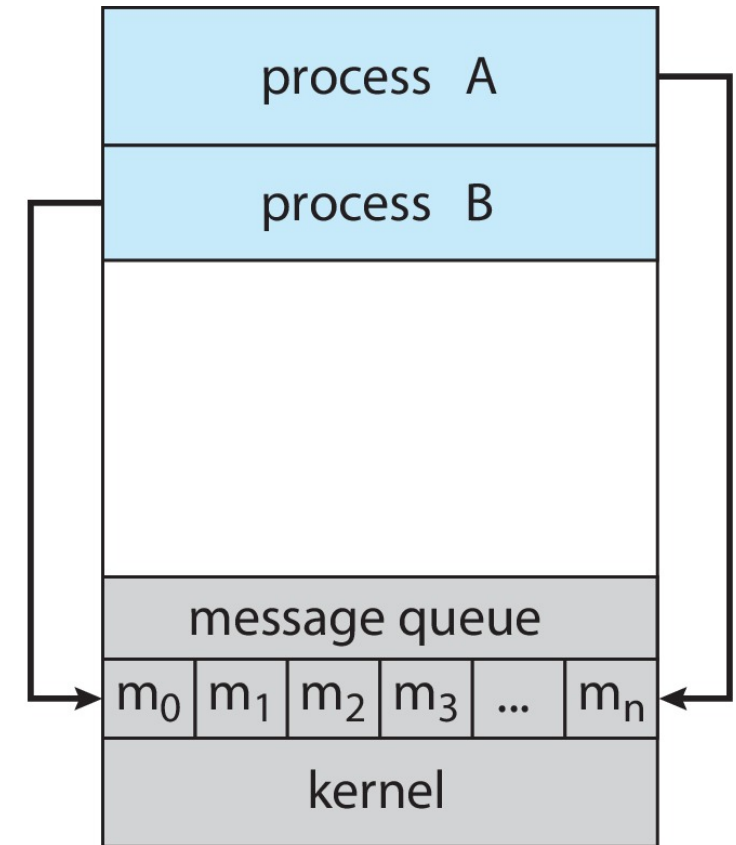
Two models of IPC

Shared memory

Message passing



(a)



(b)

- A region of memory is shared between cooperating processes.
- One process creates a **shared memory segment**.
- a shared-memory region resides in the address space of the process creating the shared-memory segment.
- Other processes that want to communicate must **attach the shared memory segment** to their address space.
- Once shared memory is established, processes can **read and write directly** in that region.

# Producer-Consumer Problem

Paradigm for cooperating processes:

- *producer* process produces information that is consumed by a *consumer* process

Two variations:

**unbounded-buffer** - places no practical limit on the size of the buffer:

- Producer never waits
- Consumer waits if there is no buffer to consume

**bounded-buffer** - assumes that there is a fixed buffer size

- Producer must wait if all buffers are full
- Consumer waits if there is no buffer to consume

# Producer-Consumer Problem



- One solution to the producer–consumer problem uses shared memory.
- To allow producer and consumer processes to run concurrently, we must have available a buffer of items that can be filled by the producer and emptied by the consumer.
- This buffer will reside in a region of memory that is shared by the producer and consumer processes.
- A producer can produce one item while the consumer is consuming another item.
- We can do so by having an integer **counter** that keeps track of the number of full buffers.
- Initially, **counter** is set to 0.

- Shared data

```
#define BUFFER_SIZE 10
typedef struct {
    . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```



# Producer Process – Shared Memory



```
while (true) {  
    /* produce an item in next produced */  
  
    while (counter == BUFFER_SIZE)  
        ; /* do nothing */  
  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```

```
while (true) {  
    while (counter == 0)  
        ; /* do nothing */  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
    /* consume the item in next consumed */  
}
```

- Message passing allows processes to communicate and synchronize their actions by **sending** and **receiving messages**.
- Unlike shared memory, processes do not share the same address space. Instead, the OS provides mechanisms (**system calls**) for safe **data exchange**.
- It is particularly useful in a distributed environment, where the communicating processes may reside on different computers connected by a network.
- A message-passing facility provides two operations:
  - **send**(*message*)
  - **receive**(*message*)
- If processes *P* and *Q* wish to communicate, they need to:
  - Establish a **communication link** between them
  - Exchange messages via **send/receive**

- Here are several methods for logically implementing a link and the send()/receive() operations:
  - **Direct or indirect communication**
  - **Synchronous or asynchronous communication**
  - **Automatic or explicit buffering**

- Each process that wants to communicate must **explicitly name** the **recipient** or **sender** of the communication.
- In this scheme, the `send()` and `receive()` primitives are defined as:
  - **`send(P, message)`**—Send a message to process P.
  - **`receive(Q, message)`**—Receive a message from process Q.

- With indirect communication, the messages are **sent** to and **received** from **mail boxes, or ports**.
- A mailbox can be an **object** into which messages can be placed by processes and from which messages can be removed
- Each mailbox has a **unique id**

## Operations:

- Create a new mailbox (port)
- Send and receive messages through mailbox
- Delete a mailbox
- Primitives are defined as:
  - **send**(*A, message*) – send a message to mailbox A
  - **receive**(*A, message*) – receive a message from mailbox A

- Message passing may be either **blocking or nonblocking**— also known as **synchronous and asynchronous**.
- **Blocking** is considered **synchronous**
  - **Blocking send** -- the sender is blocked until the message is received
  - **Blocking receive** -- the receiver is blocked until a message is available
- **Non-blocking** is considered **asynchronous**
  - **Non-blocking send** -- the sender sends the message and continue
  - **Non-blocking receive** -- the receiver receives:
    - A valid message, or
    - Null message

When both `send()` and `receive()` are blocking, we have a **rendezvous** between the sender and the receiver.

- Whether communication is direct or indirect, messages exchanged by communicating processes reside in a **temporary queue**.
- Basically, such queues can be implemented in three ways

## **Zero capacity**

- The queue has a maximum length of zero; thus, the link cannot have any messages waiting in it.
- In this case, the sender must block until the recipient receives the message.

## **Bounded capacity.**

- The queue has finite length  $n$ ; thus, at most  $n$  messages can reside in it.
- If the link is full, the sender must block until space is available in the queue.

## **Unbounded capacity –**

- infinite length
- Sender never waits



- **Producer:**

```
message next_produced;
while (true) {
    /* produce an item in next_produced */

    send(next_produced);
}
```

- **Consumer:**

```
message next_consumed;
while (true) {
    receive(next_consumed)

    /* consume the item in next_consumed */
}
```

- A **thread** is the **smallest unit of execution** in an operating system.
- A **thread is a lightweight execution unit inside a process** that allows multitasking and faster performance.
- It is like a **lightweight process**.
- A process can have **one or many threads**.
- Each thread has its own:
  - **Program Counter (PC)** – tells which instruction to execute next
  - **Registers** – hold intermediate data
  - **Stack** – used for function calls and local variables
- But **threads of the same process share**:
  - Code section
  - Data section (global variables, heap memory)
  - Open files and resources

# Process VS Thread



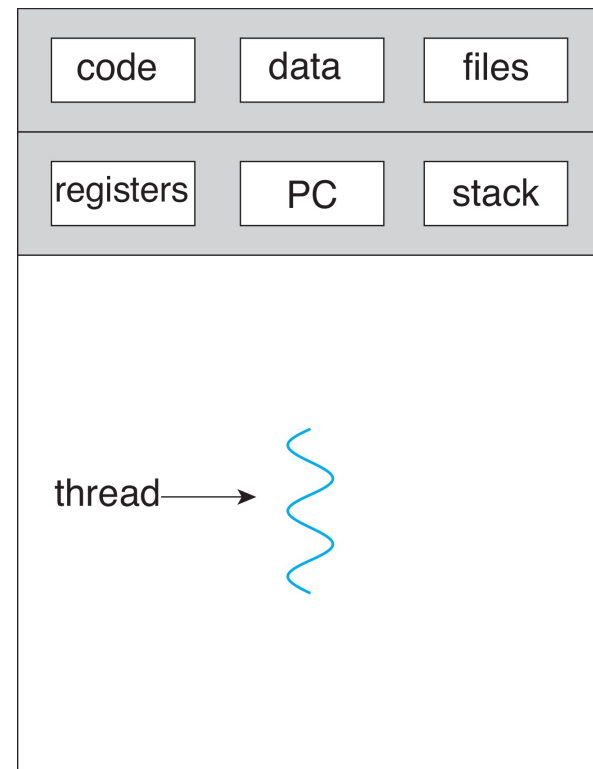
| Aspect        | Process                                                                   | Thread                                                                                                  |
|---------------|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Definition    | Independent program in execution                                          | Smallest execution unit inside a process                                                                |
| Memory        | Own memory (code, data, heap, stack)                                      | Shares process memory; own stack & registers                                                            |
| Creation      | heavy-weight                                                              | Light-weight                                                                                            |
| Communication | Needs IPC                                                                 | Easy via shared memory                                                                                  |
| Isolation     | Processes are isolated from each other (one crash doesn't affect others). | Threads are not isolated (if one thread crashes, the whole process may crash).                          |
| Example       | Chrome browser = one process per tab.                                     | Within a single Chrome tab: one thread for rendering, one for JavaScript execution, one for user input. |

# Single and Multithreaded Processes

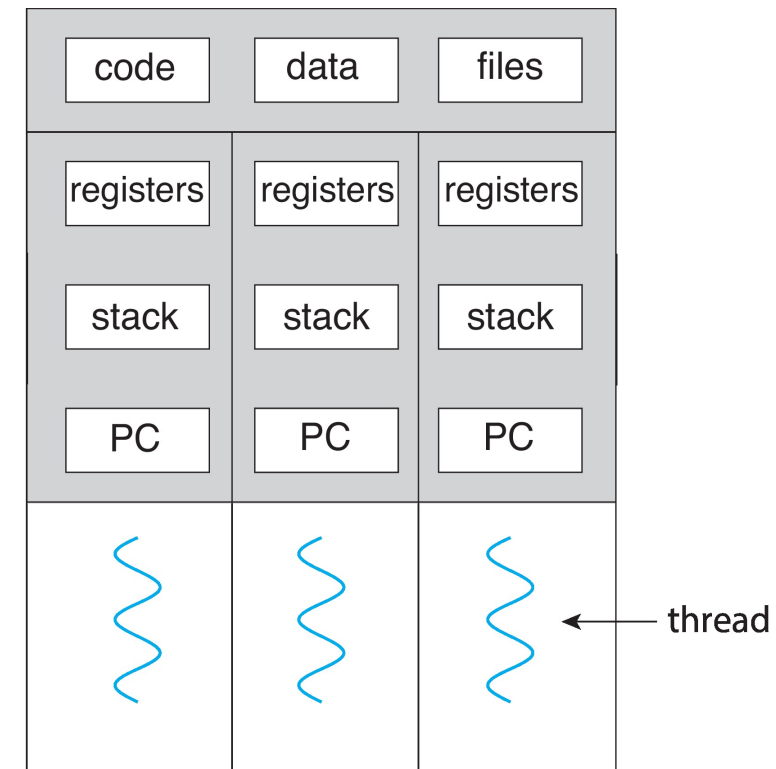


## 1. Single-Threaded Process

- Has **only one thread** of control.
- The process executes **one task at a time**
- **Example:**
- A simple text editor that allows only typing (no background saving, no spell-checking).



single-threaded process



multithreaded process

## 2. Multi-Threaded Process

- Has **two or more threads** within the same process.
- If a process has multiple threads of control, it can perform more than one task at a time.
- All threads **share the same code, data, and resources** but have separate stacks and registers.
- Most software applications that run on modern computers are multithreaded.
- For example, A word processor(doc) may have a thread for displaying graphics, another thread for responding to keystrokes from the user, and a third thread for performing spelling and grammar checking in the background.

# Types of Threads

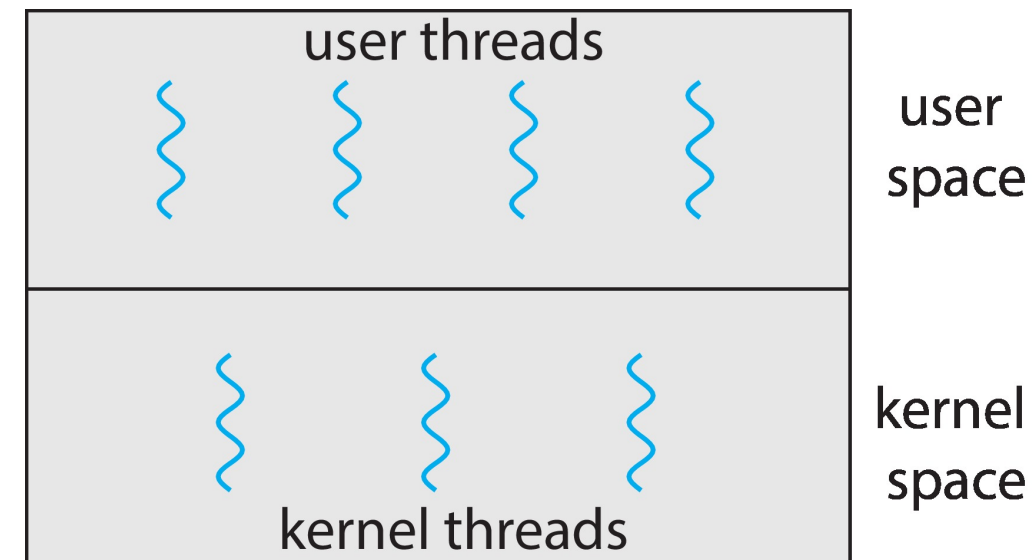


## user threads:

- Managed by the user-level threads library, not the OS.
- Faster to create and switch.

## kernel threads

- Managed by the OS.
- OS scheduler handles them.
- Slower to create/switch than user threads, but more powerful.



- To connect **user threads** to **kernel threads**, different **multithreading models** are used.
  1. Many-to-One
  2. One-to-One
  3. Many-to-Many

# Many-to-One



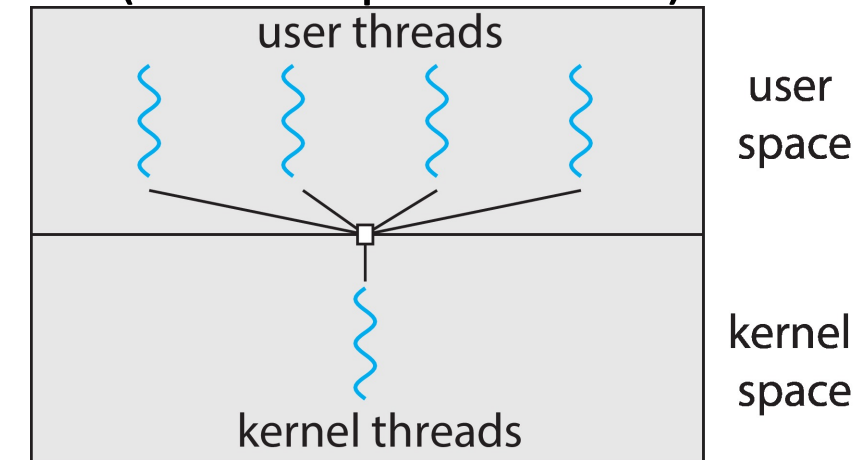
- The many-to-one model maps **many user-level threads to one kernel thread**.
- One **thread blocking** causes the **entire process to block**
- Multiple threads may not run in **parallel** because the kernel can schedule only one thread at a time.
- Few systems currently use this model

Examples:

**Solaris Green Threads**

**GNU Portable Threads**

- **Pros:** Efficient, less overhead.
- **Cons:** If one thread blocks → all threads in the process block (no true parallelism).



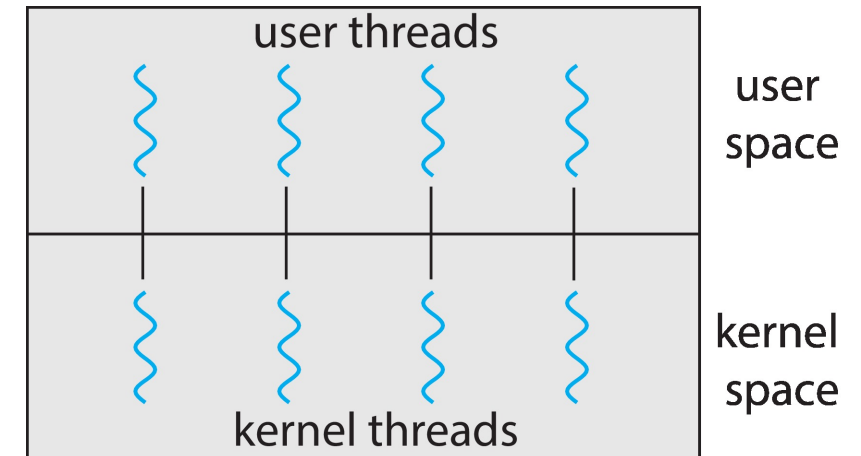


# One-to-One

- The one-to-one model maps **each user thread to a kernel thread**.
- Creating a user thread requires creating the corresponding kernel thread.
- More concurrency than many-to-one Because if one thread makes a **blocking system call**, other threads can still run.
- Multiple threads can run **simultaneously on multiprocessors**
- Number of threads per process sometimes restricted due to overhead of creating kernel threads

## Examples

- Windows
- Linux



# Many-to-Many Model



- The many-to-many model maps **many user-level threads to a smaller or equal number of kernel threads.**
- The number of kernel threads may be specific to either a particular application or a particular machine
- Developers can create as many user threads as necessary, and the corresponding kernel threads can run in parallel on a multiprocessor.
- When a thread performs a blocking system call, the kernel can schedule another thread for execution.
- **Example:** Solaris (older versions), Windows ThreadPool.

